Instituto Superior de Engenharia de Lisboa

LEIRT, LEIM, LEIC

Segurança Informática

Primeiro trabalho, Semestre de Inverno de 26/27

Entrega no Moodle até 24 de outubro de 2026

Entregar:

- documento em PDF com identificação do grupo, respostas às perguntas, comandos OpenSSL, descrição dos ficheiros produzidos a incluir em anexo, análises e experiências solicitadas;
- ficheiros em separado com código fonte nas questões que envolvem implementação (parte 2).

Parte 1 (30 pontos)

Nos sistemas Linux/macOS, a ferramenta `openssl` está normalmente disponível na linha de comandos. Poderá verificar a sua disponibilidade e versão com o comando:

```
$ openssl -version
OpenSSL 3.6.2 7 Apr 2026 (Library: OpenSSL 3.6.2 7 Apr 2026)
```

Nos sistemas Windows recomenda-se utilizar a última versão binária disponível num dos sites referidos aqui: https://github.com/openssl/openssl/wiki/Binaries. Os comandos do enunciado foram testados com a versão https://download.firedaemon.com/FireDaemon-OpenSSL/openssl-3.5.3.zip. Neste ZIP o ficheiro `openssl.exe` está na diretoria `openssl-3.5.3\x64\bin`.

```
C:\openssl\x64\bin>openssl -version
OpenSSL 3.5.8 16 Sep 2025 (Library: OpenSSL 3.5.3 16 Sep 2025)
```

1. Pretende-se garantir confidencialidade e autenticidade de um ficheiro (“alice.txt”) com o objetivo de o guardar ou enviar num canal inseguro. Considere os seguintes comandos OpenSSL [1] e a utilização de um comando para concatenar os ficheiros `g` e `h` no ficheiro `result` (`cat` em Linux/Unix [2] ou o comando `gc` em Windows [3]).

   Note que cada grupo deve sufixar o valor K com os números dos elementos do grupo, registando no relatório os valores usados. No exemplo a seguir, os números são 57627 e 57111.

   O Ficheiro “alice.txt” referido nos comandos corresponde ao fornecido junto a este enunciado.

```
$ openssl enc -e -aes-128-cbc -in alice.txt -out g \
-iv 0ed425637bb85abc97908b7abde9980a -K 42c2771f807215802dff135762757111
$ openssl mac -digest SHA256 -macopt key:changeit -in g -out h HMAC
$ cat g h > result
```

Para o caso de sistemas Windows via Powershell podem considerar, em alternativa ao `cat`, os comandos:

```
$ gc g, h -Encoding byte | sc result -Encoding Byte
```

   1.1. Foi adicionado algum *padding* durante estas operações? Explique como chegou a esta conclusão. Caso tenha sido, indique qual comando realizou a adição do *padding*, quantos e quais os bytes adicionados.

   1.2. Os comandos correspondem a um esquema *mac-then-encrypt* ou *encrypt-then-mac*? Justifique.

   1.3. Com base na documentação OpenSSL ([4, 5]) e nos exemplos presentes no repositório da disciplina [6], indique a sequência de comandos para desfazer a proteção realizada nos comandos do enunciado.

2. Considere duas partes num sistema de comunicação, Alice e Bob, bem como o esquema de assinatura digital RSA com SHA256, composto pelos algoritmos de assinatura (S) e de verificação de assinatura (V). O algoritmo S calcula internamente o resumo SHA256 da mensagem e assina esse resumo; o algoritmo V recalcula o resumo da mensagem recebida e verifica se corresponde ao que foi assinado. Assuma que KsAlice e KvAlice são, respetivamente, as chaves privada e pública de Alice, e que Bob conhece previamente KvAlice.

   Envio. Alice assina a mensagem m com a sua chave privada e envia a Bob a mensagem e a respetiva assinatura:

   ```
   s = S(KsAlice)(m)
   Alice → Bob : m || s
   ```

   Receção. Bob separa a mensagem da assinatura e verifica-a usando a chave pública de Alice:

   ```
   (m, s) = split(m || s)
   V(KvAlice)(m, s) ?= true
   ```

   Com base na documentação OpenSSL ([1]), em particular dos comandos `openssl genpkey`, `openssl pkey` e `openssl dgst`, e nos exemplos presentes no repositório da disciplina [6], responda às questões seguintes. **Em cada alínea, indique os comandos utilizados e o *output* obtido**.

   2.1. Gere uma chave privada RSA de 2048 bits para Alice (`alice_priv.pem`) e extraia dessa chave a respetiva chave pública (`alice_pub.pem`).

   2.2. Considere os ficheiros imagem_1KB.png e imagem_10MB.png em anexo. Assine cada um com a chave privada de Alice, usando SHA256, e verifique as assinaturas no papel de Bob. Que propriedade de segurança é garantida ao ser usada uma função de *hash* adequada?

   2.3. Considere o ficheiro `alice.txt`, assine-o com a chave privada de Alice (`alice_priv.pem`) e teste a robustez do esquema nas duas situações seguintes:

      2.3.1. Faça uma cópia da mensagem (`alice_mod.txt`), altere nela um único carácter e verifique a assinatura original sobre a mensagem alterada.

      2.3.2. Gere um par de chaves para Mallory (`mallory_priv.pem`, `mallory_pub.pem`), assine `alice.txt` com a chave privada de Mallory e verifique essa assinatura com a chave pública de Alice. Compare os resultados com uma verificação bem-sucedida e enquadre os resultados nas propriedades de segurança do esquema de assinatura digital.

3. Pretende-se construir uma hierarquia de certificados com três níveis: CA raiz, CA intermédia e certificado de servidor (folha). Substitua `NUM` em todos os comandos pelo número mais baixo do grupo. Em todas as alíneas, indique os comandos completos utilizados **e apresente o output obtido**. Entregue também os ficheiros `.pem` gerados, num arquivo `NUM_certs.zip`.

   Crie os seguintes três ficheiros de configurações:

   `root.cnf`

   ```
   [ req ]
   distinguished_name = dn
   x509_extensions    = ext

   [ dn ]

   [ ext ]
   basicConstraints=critical,CA:TRUE
   keyUsage=critical,keyCertSign,cRLSign
   subjectKeyIdentifier=hash
   ```

   `inter.cnf`

   ```
   [ ext ]
   basicConstraints=critical,CA:TRUE,pathlen:0
   keyUsage=critical,keyCertSign,cRLSign
   nameConstraints=critical,permitted;DNS:.isel.pt
   subjectKeyIdentifier=hash
   authorityKeyIdentifier=keyid:always
   ```

   `leaf.cnf`

   ```
   [ ext ]
   basicConstraints=critical,CA:FALSE
   keyUsage=critical,digitalSignature,keyEncipherment
   extendedKeyUsage=serverAuth
   subjectAltName=DNS:www.alunoNNNNN.isel.pt
   subjectKeyIdentifier=hash
   authorityKeyIdentifier=keyid:always
   ```

   3.1. Execute a sequência seguinte, completando as lacunas assinaladas com `___`:

   ```
   # CA raiz (RSA 4096, auto-assinada, 3650 dias)
   openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:___ -out root.key
   openssl req -new -x509 -key root.key -sha256 -days ___ -out root.pem \
     -subj "/C=PT/O=ISEL/CN=ISEL ESS Root CA $NUM" -config root.cnf
   ```

   ```
   # CA intermédia (EC prime256v1, 1825 dias, série 0x${NUM}01)
   openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:___ -out inter.key
   openssl req -new -key inter.key -out inter.csr \
     -subj "/C=PT/O=ISEL/CN=ISEL ESS Intermediate CA $NUM"
   # indique no -CA o certicado raiz e em -CAKey a chave privada do certificado raiz
   openssl x509 -req -in inter.csr -CA ___ -CAkey ___ \
     -set_serial 0x${NUM}01 -days 1825 -sha256 -extfile ___ -out inter.pem
   ```

   ```
   # Folha (RSA 2048, 90 dias, série 0x${NUM}02)
   openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:___ -out leaf.key
   openssl req -new -key leaf.key -out leaf.csr \
     -subj "/C=PT/O=ISEL/CN=www.aluno$NUM.isel.pt"
   openssl x509 -req -in leaf.csr -CA ___ -CAkey ___ \
     -set_serial 0x${NUM}02 -days 90 -sha256 -extfile ___ -out leaf.pem
   ```

3.2. Apresente o conjunto das impressões digitais (*fingerprint*) SHA256 dos três certificados, obtida com o comando `openssl x509 -in <cert> -noout -fingerprint -sha256`.

3.3. Usando o comando `openssl x509 -in <cert> -text -noout`, indique o *Signature Algorithm* de cada certificado. Explique por que motivo o da folha não corresponde ao algoritmo da chave contida nesse certificado.

3.4. Validação da cadeia:

O comando `openssl verify` valida um certificado contra uma âncora de confiança (`-CAfile`), podendo receber certificados intermédios em `-untrusted` e o nome do servidor folha em `-verify_hostname`.

Execute os testes seguintes e, para cada um, registe a mensagem obtida (incluindo o número do erro, quando existir) e explique-a numa ou duas frases:

```
# i) sem a intermédia, e depois com a intermédia
openssl verify -CAfile root.pem leaf.pem
openssl verify -CAfile root.pem -untrusted inter.pem leaf.pem

# ii) validação do nome do servidor
openssl verify -CAfile root.pem -untrusted inter.pem \
  -verify_hostname www.aluno$NUM.isel.pt leaf.pem
openssl verify -CAfile root.pem -untrusted inter.pem \
  -verify_hostname www.aluno$NUM.example.com leaf.pem
```

Parte 2 (70 pontos)

4. Utilizando a biblioteca JCA [7], desenvolva uma aplicação que armazene *strings* na forma de uma cadeia de blocos (*Blockchain*). O programa deverá receber como argumento uma *string* de tamanho arbitrário a ser gravada na cadeia. Neste caso, um novo bloco cujo conteúdo corresponde à *string* especificada deverá ser criado e anexado à cadeia. Para esta implementação, um bloco corresponde a um conjunto de 3 ficheiros (onde N corresponde ao índice do bloco):

- N.txt: ficheiro que armazena o conteúdo do bloco (i.e., a *string* especificada pelo utilizador).
- N.nce: ficheiro que armazena o *nonce* do bloco (ver detalhes abaixo).
- N.hsh: ficheiro que armazena o *hash* H_{16} do conteúdo concatenado com o *nonce* do bloco atual concatenado com o *hash* do **bloco anterior**.
  Ou seja, $H_{16\,N} = H16(string_N \,||\, nonce_N \,||\, H_{16\,N-1})$

O *hash* H_{16} corresponde à aplicação da função SHA256 sobre a mensagem, mas aproveitando apenas os primeiros 16 bits, ou seja, $H_{16}(m) = SHA256(m)_{0..15}$.

Todos os blocos são armazenados na diretoria atual. Ao criar um bloco, a aplicação deverá ler o ficheiro com o *hash* do bloco anterior e escolher como *nonce* um número tal que o *hash* do novo bloco seja menor que 1024 (valor alvo). Se não houver bloco anterior, deverá considerar o *hash* do bloco anterior como 0.

4.1. Depois de concluída a implementação, realize o seguinte experimento. Partindo de uma cadeia vazia, adicione 100 blocos com dados aleatórios. Para cada execução do programa (*i.e.*, inserção de bloco), meça o tempo de execução e o número de *nonces* tentados até o sucesso. Calcule a média destes valores.

4.2. Repita o experimento da alínea anterior, mas agora alterando o valor alvo (*i.e.*, o valor ao qual o *hash* do novo bloco é comparado para se escolher o *nonce*) de 1024 para 512. Houve alteração significativa dos resultados em relação aos da alínea anterior? Porquê?

4.3. Considere a cadeia de 100 blocos criada na alínea anterior. Coloque-se no lugar de um atacante que pretende alterar o conteúdo do quinto bloco. Como esta alteração poderia ser realizada de forma que a consistência da cadeia fosse mantida? Quanto tempo, aproximadamente, o atacante necessitaria para realizar esta alteração?

5. Realize uma aplicação de consola para proteger texto (*strings*) usando a biblioteca JCA [7], representando o resultado no formato *JSON Web Encryption* (JWE) [8]. A proteção consiste no uso de cifra/decifra autenticada, com transporte de chaves usando criptografia assimétrica, tal como descrito na secção 3.3 do RFC 7516 [8]. Considere os seguintes exemplos de utilização para proteção e desproteção:

```
$ java JWE enc <some string> <recipient certificate>
JWE token = ...
```

```
$ java JWE dec <jwe string> <recipient pfx>
Decrypted text = ...
```

Sugere-se a seguinte abordagem de desenvolvimento:

a. Realize e teste um componente para cifrar e decifrar *strings* usando o algoritmo AES em modo GCM. Este componente deve receber a *string* a cifrar/decifrar, a *string* com dados adicionais, a chave simétrica, o IV e a dimensão da marca de autenticidade. Deve ser possível distinguir a parte do criptograma que corresponde à cifra da *string* e a parte que corresponde à marca de autenticidade.

b. Realize e teste um componente para cifrar e decifrar uma chave simétrica AES usando o algoritmo RSA.

c. Realize e teste um componente para obter a chave pública de um certificado X.509 validado e uma chave privada de um *keystore*.

d. Use os componentes anteriores para realizar e testar o sistema pretendido.

Use o material criptográfico presente nos certificados e *keystores* do anexo `certificates-keys.zip`.

21 de setembro de 2026

Referências

[1] OpenSSL Docs, https://docs.openssl.org/master/man1/ (18-set-2026)

[2] Comando cat, https://man7.org/linux/man-pages/man1/cat.1.html (18-set-2026)

[3] Comando gc, https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.management/get-content (18-set-2026)

[4] OpenSSL command enc, https://docs.openssl.org/master/man1/openssl-enc/ (18-set-2026)

[5] OpenSSL command mac, https://docs.openssl.org/master/man1/openssl-mac/ (18-set-2026)

[6] SegInf Github, https://github.com/isel-deetc-computersecurity/seginf-public/ (18-set-2026)

[7] Java Cryptography Architecture, https://docs.oracle.com/en/java/javase/21/security/java-cryptography-architecture-jca-reference-guide.html (18-set-2026)

[8] RFC 7516: JSON Web Encryption (JWE), https://datatracker.ietf.org/doc/html/rfc7516, 2015